



# S.P.M College, Udantpuri

Bachelor Of Computer Application (BCA)

Part -2 (Paper-3)

– Hira Kumar

## ❖ Tokens

- A token is the smallest individual element of a C++ program that is meaningful to the compiler. Tokens act as a building blocks of a program.
- The following tokens are available in C++ keywords, identifiers, constants, variables, and operators described in this section are examples of tokens. Punctuation characters such as brackets ([ ]), braces ({ }), parentheses ( ( ) ), and commas (,) are also tokens.

### ➤ Name of tokens

- ✓ Keywords
- ✓ Identifiers
- ✓ Constants
- ✓ Variables
- ✓ Operators
- ✓ Punctuation

## ❖ Variables

- In C++ a variable is a name of memory location used to store data. A variable is a location in your computer's memory in which you can store a value and from which you can later retrieve that value.
- The value stored in a variable can change during program execution.
- When you define a variable in C++, you must tell the compiler what kind of variable it is: an integer, a character, and so forth.
- When you run your program, it is loaded into RAM from the disk file. All variables are also created in RAM.

## Declaration of Variable

Declaration will allocate memory for specified variable with garbage value.

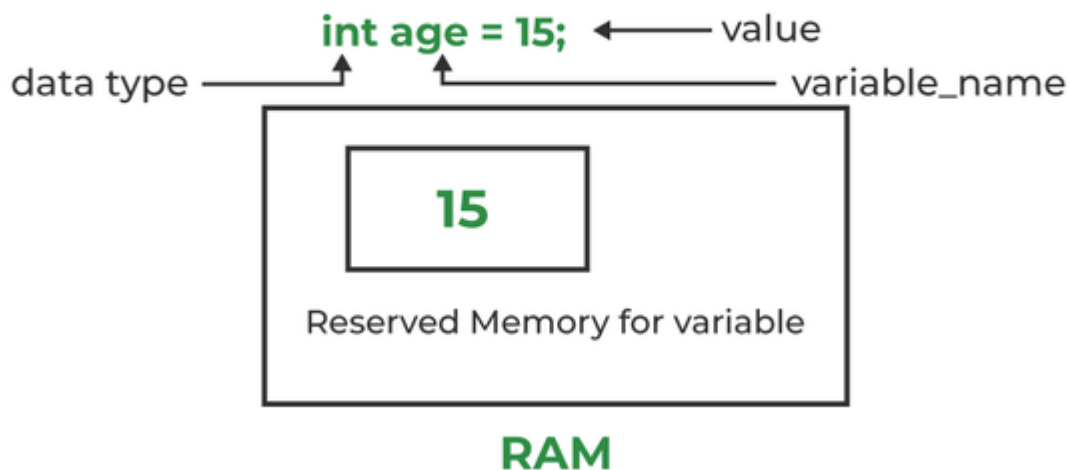
### Syntax :

```
Data-Type    Variable-name;
```

### Examples :

```
int a;  
float b;  
char c;
```

## Variable in C++



## ❖ Initialization of Variable in Program

Initialization means assigning the first value to a variable at the time of declaration. Every new value will overrides the previous value.

```

1  #include<iostream>
2  using namespace std;
3  #include<conio.h>
4  int main()
5  {
6      int a=10, A =10; //Initialization of Variable
7      cout << a << endl << A <<endl;
8      a=75, A =22.5; //Initialization of Variable
9      cout << a << endl << A <<endl;
10     int hira=2026; //Initialization of Variable
11     cout << hira;
12     float f=5.5; //Initialization of Variable
13     cout << f <<endl;
14     char c='y'; //Initialization of Variable
15     cout << c << endl;
16     string bihar = "Hira"; //Initialization of Variable
17     cout << bihar;
18
19     getch();
20 }
21

```

C:\Users\hira\Desktop\z.exe

```

10
10
75
22
20265.5
y
Hira

```

Use this code to input any string with spaces.

```

hello.cpp x
#include<iostream>
#include<string>
#include<conio.h>
using namespace std;
int main()
{
    string s;
    cout << " Enter your name : ";
    getline(cin, s);
    cout << " Thank You !" << endl <<endl;
    cout << " Your name is : " << s;
    getch();
}

```

C:\Users\hira\Desktop\hello.exe

```

Enter your name : Hira kumar
Thank You !

```

```

Your name is : Hira kumar

```

**Q. Write a program to print the user's name, age, and DOB using user input & what answer of 5/2 in c++.**

```

1  #include<iostream>
2  using namespace std;
3  #include<conio.h>
4  int main()
5  {
6      int a,d; float f; // Initialization of Variable
7      char c;          // Initialization of Variable
8      string s;        // Initialization of Variable
9      cout << " What is your name = ";
10     cin >> s;
11     cout << s << endl;;
12     cout << "What is your age = ";
13     cin >> a;
14     cout <<a << endl;
15     cout << "What is your DOB = ";
16     cin >> d;
17     cout <<d <<"\n";
18     cout << "What answer of 5/2 = "; cin >>f;
19     cout << f;
20     getch();
21 }
22

```

C:\Users\hira\Desktop\z.exe

```

What is your name = hira
hira
What is your age = 99
99
What is your DOB = 01011990
1011990
What answer of 5/2 = 2.5
2.5

```

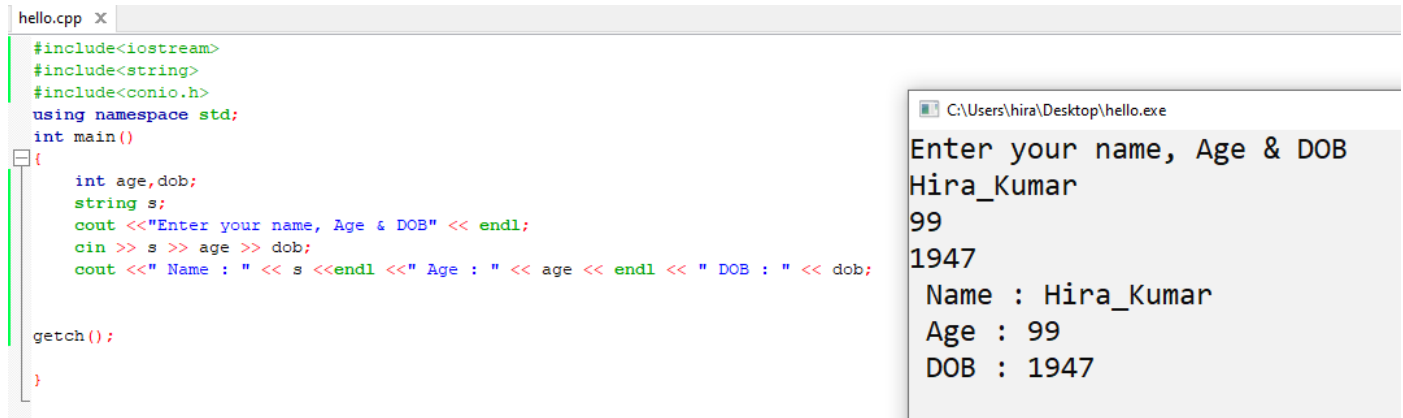
**HIRA KUMAR**

6. “Optimize or “Minimize” above c++ program. (Write a program to print name, age, DOB of user & what answer of 5/2 in c++.)

OR

Reduce the number of lines while keeping the same logic.

## Q6 Answer

The image shows a code editor window titled 'hello.cpp' with the following C++ code:

```
#include<iostream>
#include<string>
#include<conio.h>
using namespace std;
int main()
{
    int age,dob;
    string s;
    cout <<"Enter your name, Age & DOB" << endl;
    cin >> s >> age >> dob;
    cout <<" Name : " << s <<endl <<" Age : " << age << endl << " DOB : " << dob;

    getch();
}
```

Next to the code editor is a screenshot of the program's execution. The window title is 'C:\Users\hira\Desktop\hello.exe'. The output is:

```
Enter your name, Age & DOB
Hira_Kumar
99
1947
Name : Hira_Kumar
Age : 99
DOB : 1947
```

## ❖ C++ Identifiers

In C++ programming language, identifiers are the unique names assigned to variables, functions, classes, structs, or other entities within the program.

### ➤ Rule = How to write Variable\_name / Identifiers

- It must begin with alphabet or underscore. (e.g – a2, Hira, \_2, bihar\_sharif, .....)
- No other symbol is allowed (e.g – a\$, a.b, x-1,...)
- An identifie name cannot start with a digit (e.g – 2a, 8x,...)
- C++ has reserved keywords that cannot be used as identifiers.
- Identifier must be unique in its namespace.

| Valid Identifiers                                                                                                                                        | Invalid Identifiers                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Hira;</li><li>• A;</li><li>• num;</li><li>• bihar_sharif;</li><li>• a2;</li><li>• sum;</li><li>• _xyz;</li></ul> | <ul style="list-style-type: none"><li>• 1hira (start with digit)</li><li>• bihar sharif (space not allow)</li><li>• goto (keyword)</li><li>• 2a; (start with digit)</li><li>• Bihar@123 (special symbol not allow)</li></ul> |

## ❖ Keywords

- It is also called Pre-define words / Reserved word /type of token
- Keywords (reserved words) have special meanings to the C++ compiler and are always written or typed in lowercase letters.
- Keywords are words that the language uses for a special purpose, such as void, int, public, etc.
- It can't be used for a variable name or function name or any other identifiers.
- The total count of keywords is 95, in which 32 keywords present in C language.
- Categorization of c++ keywords 

### Examples of Keywords:

|          |           |           |               |
|----------|-----------|-----------|---------------|
| asm      | double    | new       | <u>switch</u> |
| auto     | else      | operator  | template      |
| break    | enum      | private   | this          |
| case     | extern    | protected | throw         |
| catch    | float     | public    | try           |
| char     | for       | register  | typedef       |
| class    | friend    | return    | union         |
| const    | goto      | short     | unsigned      |
| continue | <u>if</u> | signed    | virtual       |
| default  | inline    | sizeof    | void          |
| delete   | int       | static    | volatile      |
| do       | long      | struct    | while         |

Now- Modern C++ ( Version : C++23) has more than 90 reserved keywords.

## ❖ Constants

- Constants (often referred to as Literals) are data items that never change their value during the execution of the program.

### Type of Constants

The following types of constants are available in C++.

- ✓ Integer Constants
- ✓ Character Constants
- ✓ Floating Point Constants
- ✓ Strings Constants
- **Integer constant**- Consists of digits without a decimal point.
  - ✓ Example :- 10, -555, 10A(hexa form), 22, -9, .....
- **Float constant**- Consists of the digits with decimal.
  - ✓ Example :- 3.0, -7.5, -0.067, 45.55, 2.5e3, .....
- **Character constant**- single character enclosed with in a part of single quote.
  - ✓ Example :- 'A', '9', '#', .....
- **String constant**- Collection is sequence of characters enclosed in double quotes.
  - ✓ Example :- "Hira", "New Delhi", .....

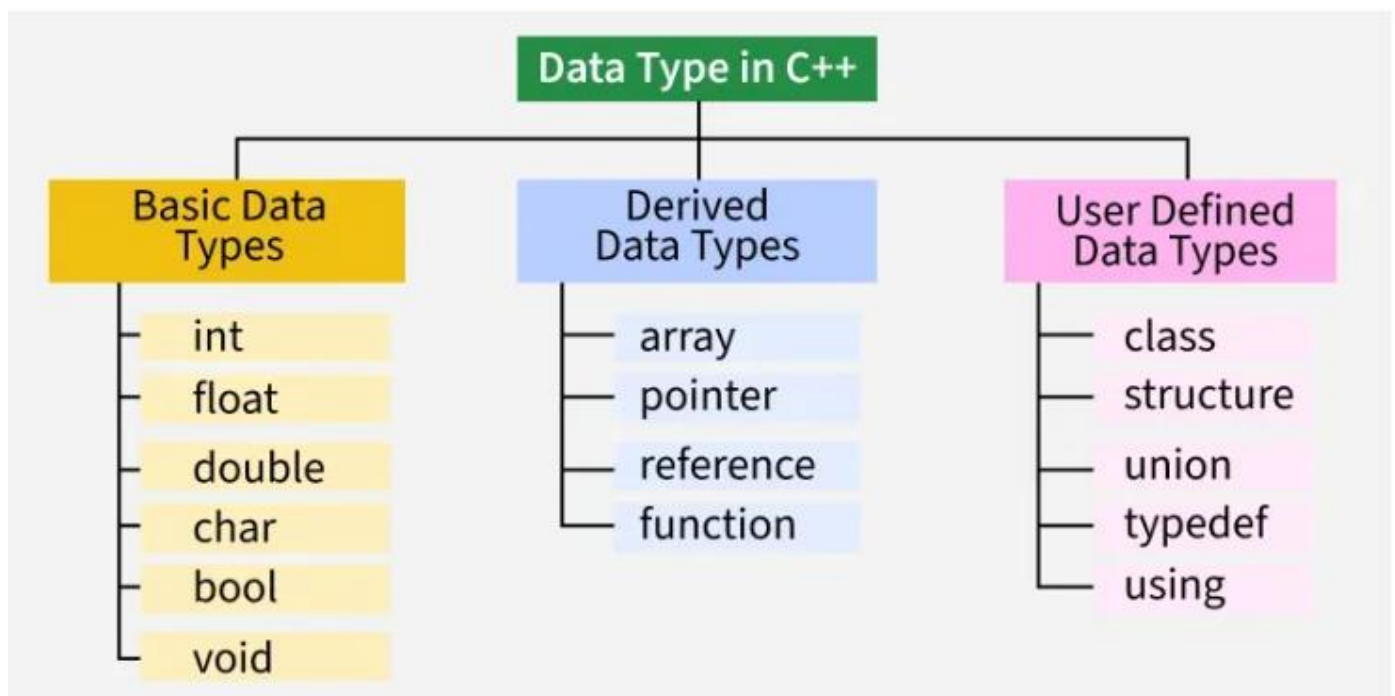
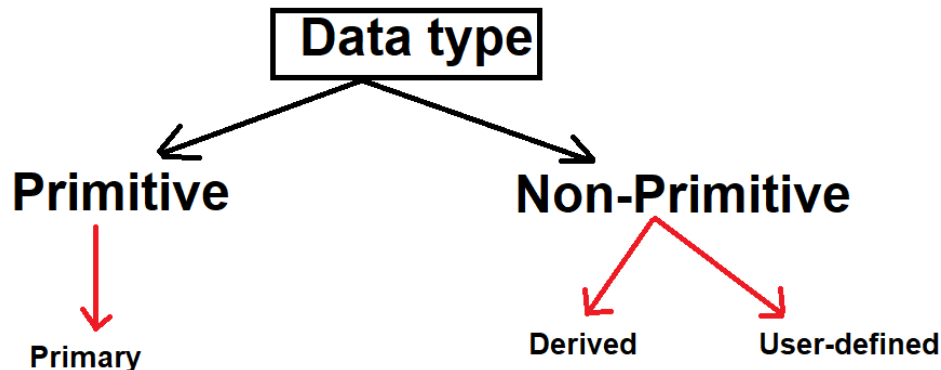
## ❖ Data Type

- The data type determines the kind of information that may be stored in the variable. Variables are classified according to their data type.
- Whenever a variable is defined in C++, the compiler allocates memory for that variable based on the data type with which it is declared.

- The programmer can select the data type appropriate to the needs of the application. Data types specify the size and types of values to be stored.

## **C++ supports the following data types:**

1. Primary or Built-in or Fundamental/Basic data type
2. Derived data types
3. User-defined data types



## ❖ **Size of Primitive Data type**

(64 bit compilers – 2025-2026)

| Data Type | Size (byte) | Use                       | Range                     |
|-----------|-------------|---------------------------|---------------------------|
| int       | 4           | To store integers numbers | -2147483648 to 2147483647 |

|             |   |                                                                |                                               |
|-------------|---|----------------------------------------------------------------|-----------------------------------------------|
| long        | 4 | For large integers numbers                                     | -2147483648 to 2147483647                     |
| long long   | 8 | For very large integers numbers                                | -9223372036854775808 to 9223372036854775807   |
| float       | 4 | For decimal numbers                                            | $-3.4 \times 10^{38}$ to $3.4 \times 10^{38}$ |
| double      | 8 | For high precision decimal numbers                             | $\pm 1.7 \times 10^{308}$                     |
| long double | 8 | For extra high precision decimal numbers (some compiler 12/16) | $\pm 1.1 \times 10^{4932}$                    |
| char        | 1 | To store a single character                                    | 'A', 'a', '0', .....                          |
| bool        | 1 | To store true/false values                                     | True/false                                    |

The screenshot shows a C++ program in a code editor (z.cpp) and its execution output in a terminal window. The program uses `sizeof` to determine the size of various data types in bytes.

```

1  #include<iostream>
2  #include<conio.h>
3  using namespace std;
4  int main()
5  {
6      // size in byte
7      cout << "Size of Int = " << sizeof(int) << endl;
8      cout << "Size of Float = " << sizeof(float) << endl;
9      cout << "Size of Char = " << sizeof(char) << endl;
10     cout << "Size of Long = " << sizeof(long) << endl;
11     cout << "Size of Long long = " << sizeof(long long) << endl;
12     cout << "Size of Double = " << sizeof(double) << endl;
13     cout << "Size of Bool = " << sizeof(bool) << endl;
14     cout << "Size of void = " << sizeof(void) << endl;
15     getch();
16     return 0;
17 }

```

The terminal output shows the following results:

```

C:\Users\hira\Desktop\z.exe
Size of Int = 4
Size of Float = 4
Size of Char = 1
Size of Long = 4
Size of Long long = 8
Size of Double = 8
Size of Bool = 1
Size of void = 1

```

## ❖ C++ Modifiers

- Modifiers are used with primary data types like int, float, char to control size, sign, or behavior.

**Data type modifiers available in C++ are:**

- ✓ Signed → Can store positive & negative values
- ✓ Unsigned → Can store only positive values



- ✓ Short → Reduces the size of integer type
- ✓ Long → increases the size of integer type

### Modifiers in C++



### Old 16bit /Turbo C++ sizes

| Type               | Size (bits) | Size (bytes) | Range                   |
|--------------------|-------------|--------------|-------------------------|
| char               | 8           | 1            | -128 to 127             |
| unsigned char      | 8           | 1            | 0 to 255                |
| int                | 16          | 2            | $-2^{15}$ to $2^{15}-1$ |
| unsigned int       | 16          | 2            | 0 to $2^{16}-1$         |
| short int          | 8           | 1            | -128 to 127             |
| unsigned short int | 8           | 1            | 0 to 255                |
| long int           | 32          | 4            | $-2^{31}$ to $2^{31}-1$ |
| unsigned long int  | 32          | 4            | 0 to $2^{32}-1$         |
| float              | 32          | 4            | 3.4E-38 to 3.4E+38      |
| double             | 64          | 8            | 1.7E-308 to 1.7E+308    |
| long double        | 80          | 10           | 3.4E-4932 to 1.1E+4932  |

## Revision 1<sup>st</sup> year

### ❖ What is Sign & unsign integer

- **Signed integer** : A signed integer can store both positive(+ve) and negative (-ve) number. The leftmost bit (MSB) is used as the sign bit.

**0 = positive**

**1 = negative**

Example : **4 bit signed integer**

| <b>1 = negative</b>                                                               | <b>0 = Positive</b>                                                               |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
| <b>1000 = -8</b>                                                                  | <b>0000 = 0</b>                                                                   |
| <b>1001 = -7</b>                                                                  | <b>0001 = 1</b>                                                                   |
| <b>1010 = -6</b>                                                                  | <b>0010 = 2</b>                                                                   |
| <b>1011 = -5</b>                                                                  | <b>0011 = 3</b>                                                                   |
| <b>1100 = -4</b>                                                                  | <b>0100 = 4</b>                                                                   |
| <b>1101 = -3</b>                                                                  | <b>0101 = 5</b>                                                                   |
| <b>1110 = -2</b>                                                                  | <b>0110 = 6</b>                                                                   |
| <b>1111 = -1</b>                                                                  | <b>0111 = 7</b>                                                                   |

- **Unsigned Integer** : An unsigned integer can store only non-negative (0 and positive) numbers.

Example : **4 bit unsigned binary**

0 = Positive



0000 = 0

0001 = 1

0010 = 2

0011 = 3

0100 = 4

0101 = 5

0110 = 6

0111 = 7

0 = Positive



1000 = 8

1001 = 9

1010 = 10

1011 = 11

1100 = 12

1101 = 13

1110 = 14

1111 = 15

hello.cpp X

```
#include<iostream>
#include<conio.h>
using namespace std;
int main()
{
    int a = 2147483647; // signed integer use
    cout << "value of a = " << a << endl;
    int b = 2147483650; // over limit
    cout << "value of b = " << b << endl;
    unsigned int c = 2147483650; // unsigned used
    cout << "value of c = " << c << endl;

    getch();
}
```

C:\Users\hira\Desktop\hello.exe

```
value of a = 2147483647
value of b = -2147483646
value of c = 2147483650
```

=====HIRA KUMAR=====